N-Body Simulation

1. Overview

In this assignment, you will implement an *n-body simulation*, which is a prediction of the movements of a large number of particles that interact with one another gravitationally. You will need to finish implementing a sequential version of this simulation and study the performance of the sequential version. Then you will be directed to use OpenMP to improve the programs' performance through shared memory parallelism.

You will need to instrument your code to determine where the most time is spent in computation and evaluate where optimizations are most valuable. You will also need to focus on avoiding sequential bottlenecks, memory contention, and workload imbalance.

1.1 Environment

This is a CPU assignment where you are expected to complete the program using OpenMP and can evaluate on the Istanbul test server. For this assignment, where performance is a major portion of what you are attempting to quantify and achieve, it is unlikely running locally on your machine with presumably only a couple of cores will demonstrate the acceleration for which we are looking.

For grading evaluation, we will evaluate your performance using a 48-core AMD Epyc server (Istanbul)

1.2 Resources

For this assignment, our lecture slides along with the available documentation from the OpenMP foundation will be valuable. As a reminder, our course slides may be found on the course website, and links to OpenMP resources may be found in the course syllabus. It may also be beneficial to review C++ smart pointers, as they will be used to build your acceleration data structure.

2. Introduction

In this problem, we will simulate the movement of particles with varying mass and initial position. Each particle is defined as a tuple of id, mass, initial velocity, and initial position. Particles are affected by gravitational forces from nearby particles.

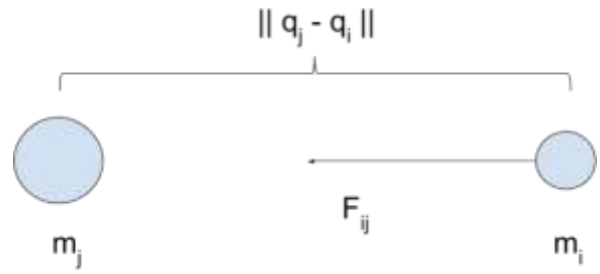

Figure 1: Illustration of Gravitational Force Between Two Particles

The *n*-body problem considers *n* point masses m_i, $i = 1,2, \ldots, n$ in an inertial frame of reference in three-dimensional space R^3 moving under the influence of mutual gravitational attraction. Each mass m has a position of vector q_i. Newton's second law says that mass times acceleration $m_i \frac{d^2 * q_i}{dt^2}$ is equal to the sum of the forces on the mass. Newton's lar of gravity says that the gravitational force felt on mass m_i is given by

$$\boldsymbol{F}_{i,j} = \frac{G m_i m_j \, (\boldsymbol{q}_j - \boldsymbol{q}_i)}{\|\boldsymbol{q}_j - \boldsymbol{q}_i\|}$$

where G is the gravitational constant and $\|\boldsymbol{q}_j - \boldsymbol{q}_i\|$ is the magnitude of the distance between q_i and q_j (metric induced by the l_2 norm).

In the simulation you will write, particles only exist in a 2D world, with x and y dimensions. We will also use an approximation; past a predefined radius, particles will not affect other particles, since their gravitational force is almost negligible.

2.1 Computing Gravitational Force

In this assignment, you do not need to implement the physical formula that defines the number of gravitational forces. We provide a function, `computeForce(Particle p1, Particle p2, float radius)`, in the file `world.h` that returns the gravitational force between particle p1 and p2. You are not allowed to modify this function. It is summarized as the following;

```
computeForce(Particle target, Particle attractor, float radius) =
       dist = distance between particles
       dir = direction from target to attractor particle
       force = dir * target mass * attractor mass * (Gravity / (dist *
              dist));
       return force
```

The `radius` parameter is a predefined input. The `computeForce` function is guaranteed to return 0 if the distance between p1 and p2 is greater than `the radius`.

2.2 Simple Simulation Algorithm

We provide a simple sequential simulation algorithm, which has no performance optimizations, called "simple simulator." It computes the total force on every particle pi by summing the individual forces that result from the gravitational interactions of pi and every nearby particle pi. However, it only considers particles that are within a certain radius of pi, because particles outside of that radius have a negligible force effect.

The pseudo-code for this implementation is as follows:

```
for each iteration:
       for each particle p:
              find nearby particles whose distance to this particle is
                     less than 'radius'.
              // compute the sum of gravitational forces that need to
                     be applied to this particle
              force = 0
              for each nearby particle p1:
                     force += computeForce(p, p1)
              Update the position and velocity of this particle using
                     the computed force.
```

This algorithm is implemented in `src/simple-simulator.cpp` and is available for your reference. The accuracy of your quad-based sequential and parallel implementations will be checked against the output generated by this code.

The interesting part of this problem is finding the nearby particles of a given particle efficiently. A naïve approach, implemented in `src/simple-simulator.cpp`, is to test the distance of all particles in the scene. This yields an implementation with $O(n^2)$ computational complexity, where n is the number of particles in the scene. When n is very

large (e.g. 1 million), the naïve approach will take a very long time to finish. We can improve this significantly.

3. Quad-Tree Simulation Algorithm

To more efficiently find the number of 'near' particles to a specific particle *p*, we can create a tree structure to quickly determine which particles are potentially in the 'radius' of pi. In this case, we will be using a simple tree structure called a 'quad-tree.'

3.1 Quad Tree

A Quad Tree is a tree data structure in which each node has exactly four children. They are useful because they can be used to partition a two-dimensional space by recursively dividing into 4 sub-spaces.

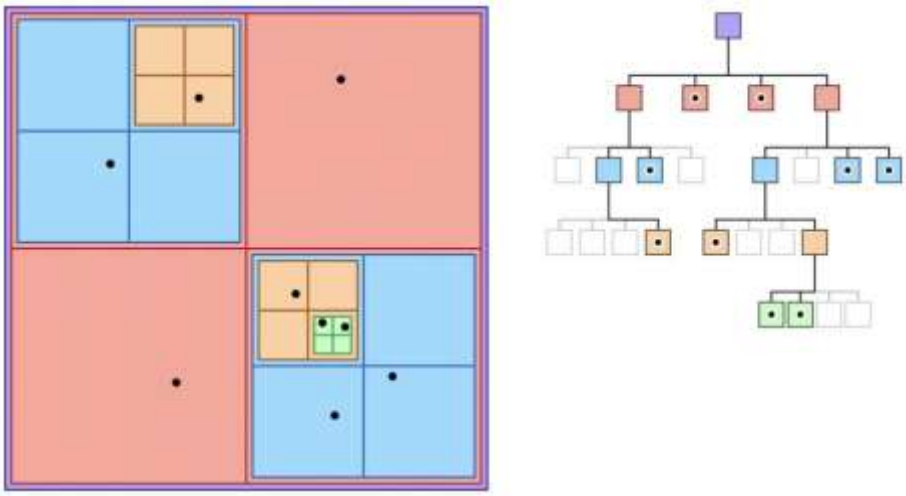

Figure 2: A Quad Tree with 'QuadTreeLeafSize' = 1

For this n-body simulation, we are interested in partitioning our 2D space into rectangles, such that each 'leaf' of the tree constains at most 'QuadTreeLeafSize' particles. Given a list of particles and their positions, we can build this quad tree by recursively bisecting the space containing those particles into nodes, while keeping track of the current square of space used. For each node, we must store the following:

1. isLeaf: whether the node is a leaf
2. particles: what particles that node contains
3. children: the four child nodes of that node

For your reference the tree building process for particles in a 2D space might look something like this:

```
QuadTreeLeafSize = max number of particles per node
buildQuadTree(particles, range_min, range_max) {
	if num(particles) <= QuadTreeLeafSize:
		return leaf node
	else:
		result = non-leaf node
		pivot = (range_min + rang_mix) *. 0.5f;
		<assign particles to appropriate children of result>
		return result
```

Note that in this algorithm, a lead node may contain 0 particles. The pseudocode is purposefully vague, as the implementation of the 2D version is left to you in Task 1.

3.2 Simulation Algorithm

When using a quad-tree structure, each simulation iteration is separated into two stages: building the quad-tree and using the quad-tree for simulation:

```
for each iteration
	1. Build quad-tree
	2. for each particle p1
		find nearby particles using the quad-tree
		compute the sum of gravitational forces that need to be applied
			to this particle
		force = 0;
		for each nearby particle p1
			force += computeForce(p, p1)
		update the position and velocity of this particle using the
			computed force
```

In this assignment, you will first finish implementing a sequential version of this simulation by completing a function building a quad-tree, then parallelizing both the tree construction and simulation stage.

4. Starter Code

We provide starter code, which includes the simple simulator and an incomplete version of the sequential tree-based algorithm.

4.1 Building the Starter Code

After downloading the starter code, you can the starter code using:

```
$ make all
```

This will build an executable named `nbody-release`. To perform a debug build, run:

```
$ make all CONFIGURATION=debug
```

4.2 Running the Startup Code

To run a simulation, use the following command:

```
$ ./nbody-release -i 10 -n 100
```

This will create a scene with 100 randomly positioned particles and run for 10 iterations. The resulting particle positions and velocities will be dumped to `out.txt` by default. You can use `-o out.txt` command-line option to change the output destination.

4.3 Visualization

You can have `nbody-release` output a visualization of the particles after each iteration. This visualization will be a 2D image of particle positions, formatted as a bitmap image file (.bmp), which is viewable in most image editors. To create a visualization, use the command-line option `-fo`:

```
$ ./nbody-release -i 10 -n 100 -fo ./
```

This informs `nbody-release` to output bmp files to the current directory that visualizes each simulation iteration. By default, the visualization represents a viewport of $-10 \leq x \leq 10$ and $-10 \leq y \leq 10$. To change the viewport size, use `-v` option:

```
$ ./nbody-release -i 10 -n 100 -fo ./ -v 20.0
```

This changes the viewport to visualize the $-20 \leq x \leq 20$ and $-20 \leq y \leq 20$ range.

4.4 Running the Benchmark

Running the following command will start a benchmark of your sequential and parallel implementations:

```
$ ./nbody-release -b
```

The benchmark will time your implementation on a variety of scenes ranging from 100 to 100,000 particles. It will also check the correctness of your simulation by comparing your simulation results to the reference results.

4.5 Usage

`nbody-release` has both testing capabilities and 'custom run' capabilities. A custom run creates a scene with 10 clusters of particles, with a number of options. The full usage of `nbody-release` is as follows:

Custom Run Options:

- `-i <int>`: This flag specifies the number of iterations, a positive integer 1 or greater, for a custom run. The default value is 1 iteration.
- `-n <int>`: Specifies the number of particles to generate.
- `-s <float>`: This flag specifies the space size that the particles for a custom run will be generated in. The default is 10.0, for which particles may have x positions in the range -10.0, 10.0 and y positions in the range -10.0, 10.0.
- `-in <string>`: Specifies an input file to read particle positions from a custom run. The file must be a .txt type file with the format of mass, x position, y position, x velocity, y velocity, space separated, with one line corresponding to each particle.
- `-o <string>`: Specifies the output file to write results. The default is out.txt.
- `-fo <string>`: When this flag is included, a .bmp visualization file will be created for each step of the simulation, labeled by step number, in the directory `<string>`. For example, `-fo "./"` will create bmp files in the same directory as `nbody-release`.
- `-v <float>`: Specifies the viewport size, which is the range of space the visualization file will use. Defaults to 10.0; in general, try to keep this value to the space bounds specified with `-s`.
- `-seq`: Runs the sequential quad-tree version of the simulation. -par: Runs the parallel quad-tree version of the simulation. -simple: Runs the provided simple version of the simulation.
- `-par`: Runs the parallel quad-tree version of the simulation
- `-simple`: Runs the provided simple version of the simulation

- `-c`: The addition of this flag specifies to run the correctness check suite for the specified implementation (-seq or -par). If none are specified, the correctness of the sequential implementation is checked by default

5. Your Tasks

Your tasks will be to complete the sequential tree-based implementation and to implement a parallel simulation implementation.

The file simple-simulator.cpp provides the naïve implementation that checks all particles for proximity. Make sure you understand this code before proceeding to Step 1.

5.1 Task 1: (15 Points) Implement Sequential Quad-Tree Based Simulation

The file seq-simulator.cpp provides the function definitions for the sequential simulation. You need to implement the `buildAccelerationStructure(..)` and `simulateStep(..)` function.

The `buildAccelerationStructure(..)` function takes a vector of current particles and should return a `QuadTree` object. See `quad-tree.h` for the definition of `QuadTree` class. You may not delete or modify the contents of this file, but you may define new functions or structures if you believe them to be useful.

A `QuadTree` has the following properties:

- `root`: a `QuadTreeNode`
- `bmin`: a Vec2 with <x,y> minimum bounds on the locations of particles in the tree
- `bmax`: a Vec2 with <x,y> maximum bounds on the locations of particles in the tree
- `getParticles`: a function that assigns the list of particles in this tree to a pointer provided as a parameter

A `QuadTreeNode` has the following properties:

- `isLeaf`: boolean labeling if this node is a leaf
- `children`: array of 4 `QuadTreeNode` pointers representing the children of this node
- `particles`: list of particles as this node

The `simulateStep(..)` function implements stage 2 of the simulation. It reads particle information from the `particles` parameter and stores the new particle status in the `newParticles` vector. It should be very similar to simple implementation.

Evaluation

Your sequential algorithm will be evaluated for its correctness. You can check the correctness of your sequential quad-tree build implementation and `simulateStep` function by running:

```
$ make
$ ./checker.pl -s
```

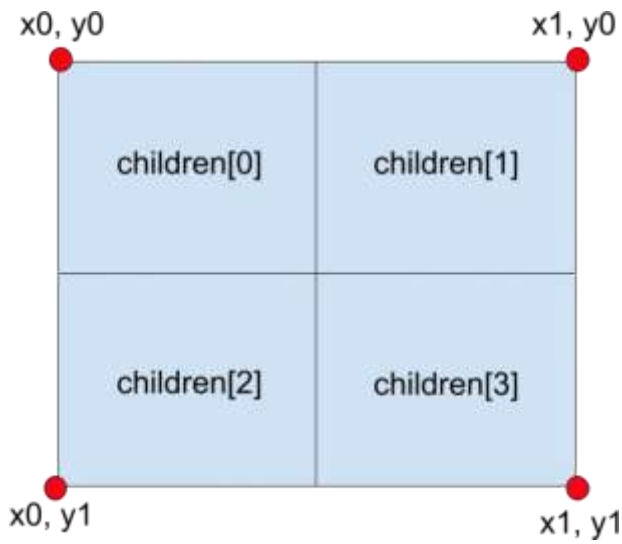

Figure 3: Diagram of Children Mapping of Quad Tree

5.2 Task 2: (25 Points) Implement Parallel Quad-Tree Based Simulation

For this task, you need to implement the `buildAccelerationStructure(..)` and `simulateStep(..)` functions in `parallel-simulator.cpp`. You may find it useful to start your sequential implementation from Task 1.

Your implementation must perform well even in scenes with imbalanced particle distribution. It will be tested against a total of 7 scenes, each shown in the gallery at the end of this writeup. It will benefit you to look at these scenes and consider them when writing your implementation.

Tips and Tricks

- Consider the use of task-based instead of data-based parallelism. What task best suits which type of parallelism? Blindly create #pragma omp parallel for loops will not serve you well in this task

- Research the different types of scheduling OpenMP provides and choose the type that best suits the problem at hand. Our slides are a good place to begin but are by no means comprehensive.

Evaluation

The benchmark shows the performance of your parallel implementation, including speedup information and correctness of errors.

```
$ make
$ ./checker.pl -p
```

You will be evaluated on both correctness and speed; an implementation that is not correct will not receive any points, so ensure that correctness does not fail. Although you could improve the apparent speedup by detuning the single-threaded performance, that will not lead to an optimal overall score, as we will look at your code. As a special case, if your single-core performance exceeds a provided baseline, then the baseline will be used as the denominator in the speedup calculation. This means that you don't need to artificially slow down a fast, single-threaded version to maximize the speedup measurement.

Scoring your performance will be 20 points for `buildAccelerationStructure` and 10 points for `simulateStep`. In this next week, your speedup targets for each of the benchmarks will be published (I am withholding this for now to see how progress on the assignment goes early on such that I establish fair targets).

5.3 Task 3: Write-up (10 points)

In addition to the code deliverables, you will also submit a half-page writeup. In this writeup, you should describe the implementation choices you made for the sequential and parallel implementations. As in our first assignment, discuss this in terms of the stages of developing a parallel program. Please include descriptions of auxiliary data structures and any other additional methods you used to improve performance.

6. Submission

Both the code and writeup will be submitted to Canvas. The writeup should be submitted as a PDF, while the code should be submitted using the file created by the following command:

```
$ make handin.tar
```

This will make an archive of your code in src in the format I am expecting for evaluation.

7. Gallery

Your implementation will be tested against the following scenes. They all have different distributions of points, different space sizes. Considering possibly uneven distributions may help improve your implementations' performance.

The red lines overlaid on the following images visualize the quad-tree based partition of the particles.

N = Number of particles in Scene

S = Space Size, where particles can take any position with x coordinate in (-S, S) and y coordinate in (-S, S).

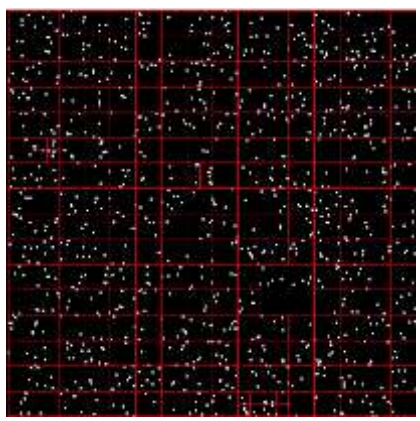

Figure 4: Random-1000: N=1000, S=10.0, randomly distributed points

Figure 5: Random-10000: N=10000,S=100.0, randomly distributed points

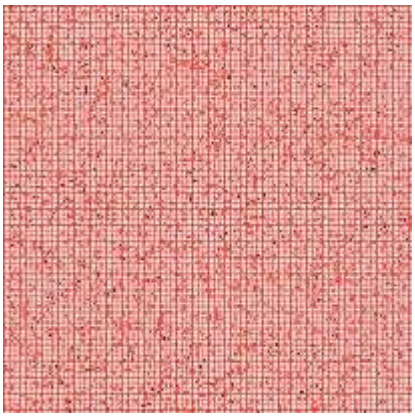

Figure 6: Random-50000: N=50000,S=500.0, randomly distributed points

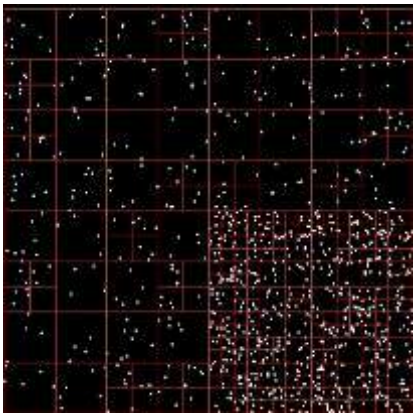

Figure 7: Corner-100: N=1000, S=10.0, corner associated points

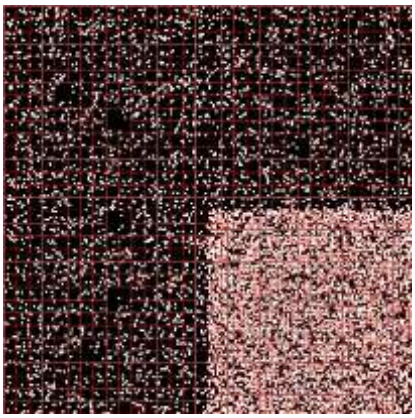

Figure 8: Corner-10000: N=10000, S=100.0, corner associated points

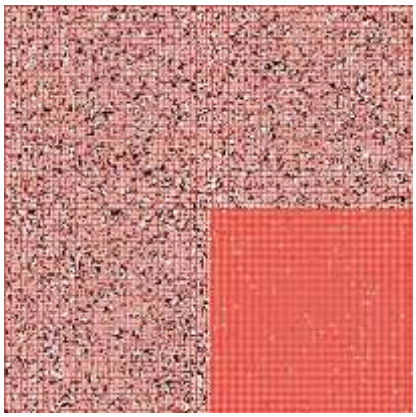

Figure 9: Corner-50000: N=50000, S=500.0, corner associated points

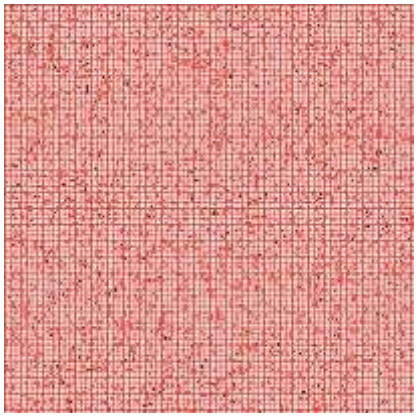

Figure 10: Repeat-10000: N=10000, S=100.0, random distributions with many iterations (50)